

8. Project Management

8. Project Management

Hai già visto Agile, Scrum e Kanban: sai quindi **come** un team organizza il lavoro giorno per giorno (sprint, board, colonne, cerimonie). Questa sezione affronta un livello diverso, complementare: quello del **Project Management**, cioè l'insieme di strumenti e pratiche che permettono di tenere sotto controllo un progetto nel suo complesso — chi è coinvolto, cosa può andare storto, a che punto siamo, e come lo comunichiamo a chi non è nella daily standup ogni giorno.

Attenzione a un equivoco comune: il Project Management **non è** la pianificazione rigida “a cascata” (Waterfall) che magari hai studiato all'università, con Gantt chart bloccati e fasi sequenziali immutabili. In un contesto Agile/DevOps, il project management convive con l'iterazione continua e l'incertezza: si pianifica, sì, ma a più livelli e con la consapevolezza che il piano cambierà. Il ruolo del Project Manager (o di chi ne fa le funzioni, come uno Scrum Master “aumentato” o un Delivery Manager) è più simile a un **navigatore che aggiorna la rotta ogni giorno** che a un ingegnere che disegna un progetto immutabile su carta.

Anche in questa sezione useremo come filo conduttore il team di **ShopFacile**, la piattaforma e-commerce che hai già incontrato: **Sara** (Product Owner) sarà la voce del business tra gli stakeholder, **Luca** (Scrum Master) seguirà da vicino processo e rischi — RAID Log, RACI, monitoraggio — mentre **Marco**, **Giulia** e **Ahmed**, il team di sviluppo, daranno concretezza agli esempi di KPI e reportistica.

Obiettivi della sezione

Alla fine di questa sezione saprai:

- identificare gli **stakeholder** di un progetto software e capire perché vanno mappati;
 - usare un **RAID Log** per tenere traccia di rischi, assunzioni, problemi e dipendenze;
 - leggere e costruire una **matrice RACI** per chiarire le responsabilità;
 - distinguere una **milestone** da una semplice scadenza;
 - valutare un **rischio** in termini di probabilità e impatto, e capire come si mitiga;
 - capire come funziona la **pianificazione a più livelli** in un contesto Agile (roadmap, release plan, sprint plan);
 - riconoscere gli strumenti di **monitoraggio** dell'andamento di un team (burndown/burnup, stand-up, dashboard);
 - conoscere i principali **KPI** usati in un team Agile/DevOps;
 - distinguere i diversi livelli di **reportistica**, a seconda di chi la legge.
-

8.1 Stakeholder: chi ha interesse nel progetto

Uno **stakeholder** (letteralmente “portatore di interesse”) è **qualsiasi persona o gruppo che può influenzare il progetto, oppure che ne è influenzato**. Non sono solo “quelli che pagano”: sono tutti quelli che, in un modo o nell’altro, hanno voce in capitolo o subiscono le conseguenze delle decisioni prese.

Analogia: organizzare un progetto software senza mappare gli stakeholder è come organizzare un matrimonio pensando solo agli sposi. In realtà ci sono genitori che pagano parte del conto, invitati con esigenze alimentari particolari, il fotografo che deve essere informato sugli orari, il comune che deve validare i documenti. Se ignori qualcuno di questi soggetti, il giorno dell’evento emergono problemi che potevi prevedere con un minimo di mappatura.

In un progetto software tipico, gli stakeholder principali sono:

- **Il cliente** (o il committente): chi ha commissionato il progetto e ne finanzia lo sviluppo. Vuole vedere valore consegnato, rispetto dei tempi e dei costi concordati.
- **Il management** (interno all’azienda che sviluppa): responsabili di linea, direttori, chi deve rendere conto a sua volta di come vanno i progetti del team.
- **Gli utenti finali:** le persone che useranno concretamente il software ogni giorno. Spesso non partecipano alle decisioni, ma sono chi subisce più direttamente le conseguenze di scelte sbagliate (un’interfaccia scomoda, una funzionalità mancante).
- **Il team:** sviluppatori, tester, designer, chiunque contribuisca alla realizzazione. Sono stakeholder anche loro: un carico di lavoro insostenibile o requisiti ambigui li riguardano direttamente.
- **Altri team o reparti:** un team di sicurezza che deve validare il rilascio, un team di infrastruttura che gestisce i server, un ufficio legale che deve approvare termini contrattuali.

Una mappatura semplice, spesso usata nella pratica, classifica gli stakeholder in base a due dimensioni: **quanto potere/influenza hanno** sul progetto e **quanto interesse** hanno nel suo esito.

Diagramma 1

Diagramma 1

Esempio pratico: nel progetto ShopFacile per una nuova funzionalità di pagamento online, il cliente/committente ha alto potere e alto interesse (va coinvolto attivamente in ogni decisione importante) — nella pratica di ShopFacile è **Sara**, la Product Owner, a rappresentare questa voce del business nelle decisioni quotidiane; il team di sicurezza interno ha alto potere ma interesse più episodico (va coinvolto nei momenti critici, come la validazione prima del rilascio in produzione); gli utenti finali hanno alto interesse ma poco potere diretto sulle decisioni (vanno rappresentati tramite ricerche utente, feedback, test di usabilità).

Perché questo conta per un Junior Project Manager? Perché la causa più comune di problemi in un progetto non è “il codice non funziona”, ma **qualcuno importante non è stato informato o coinvolto al momento giusto**. Mappare gli stakeholder all’inizio ti evita sorprese a metà progetto — ma sapere *chi* coinvolgere non basta se non hai anche un posto dove tracciare cosa può andare storto: è il compito del RAID Log, che vediamo subito.

8.2 RAID Log: la mappa dei problemi (prima che diventino problemi)

RAID è un acronimo che raggruppa quattro categorie di informazioni che un team di progetto deve tracciare costantemente:

- **R — Risks** (Rischi): eventi che *potrebbero* accadere e che, se accadono, avrebbero un impatto negativo sul progetto.
- **A — Assumptions** (Assunzioni): cose che stiamo dando per scontate, senza averle verificate al 100%, per poter procedere con la pianificazione.
- **I — Issues** (Problemi): cose che *sono già accadute* e stanno attivamente impattando il progetto ora.
- **D — Dependencies** (Dipendenze): elementi esterni al controllo diretto del team, di cui il progetto ha bisogno per procedere.

Analogia: il RAID Log è come il cruscotto di un’automobile durante un lungo viaggio. Il rischio è la spia della benzina che si accende prima che tu resti a secco (un avviso preventivo). L’assunzione è il presupposto “ci sarà un distributore aperto lungo la strada” — non l’hai verificato, ma stai viaggiando come se fosse vero. Il problema è il pneumatico che si è già bucato: è già successo, va gestito subito. La dipendenza è il fatto che, per attraversare un ponte, devi aspettare che apra alle 8, che tu lo voglia o no: non lo controlli, ma ne dipendi.

Un esempio concreto di ciascuna categoria, per un progetto software reale:

Categoria	Esempio concreto
Risk	“C'è il rischio che il carico di traffico previsto per il Black Friday superi la capacità attuale dei server, causando lentezza o down del sito.” (non è ancora successo, ma è possibile)
Assumption	“Stiamo pianificando la release assumendo che l'ambiente di test resti disponibile per tutto il mese; non abbiamo una conferma scritta dal team infrastruttura.”
Issue	“Il servizio di invio email di terze parti è down da ieri: gli utenti non ricevono le email di conferma registrazione.” (è già un problema attivo, in corso)
Dependency	“Il rilascio della nuova funzionalità di pagamento dipende dall'approvazione di sicurezza da parte di un team esterno al progetto, previsto per la settimana prossima.”

Nella pratica, un RAID Log è semplicemente una tabella condivisa (spesso in un foglio, in una board dedicata, o in strumenti come Azure DevOps, che vedremo nella sezione 10), aggiornata regolarmente, con colonne tipo: descrizione, categoria, responsabile, impatto, stato, data di apertura, azione prevista.

Un aspetto importante da capire: le **assunzioni**, se non verificate per tempo, diventano facilmente **rischi**; e i **rischi**, se non mitigati per tempo, diventano **issue** attive. Il RAID Log serve proprio a intercettare questa “scala di gravità” prima che il problema esploda a valle.

Diagramma 2

Diagramma 2

Esempio pratico: un estratto reale (semplificato) di RAID Log a metà dello Sprint 6 di ShopFacile, così come potrebbe apparire in una board condivisa con il team. In ShopFacile non esiste un Project Manager dedicato: le responsabilità di “Tech Lead” e “Project Manager” viste in tabella sono spesso portate avanti da **Luca**, insieme al team, accanto al suo ruolo di Scrum Master:

ID	Categoria	Descrizione	Responsabile	Impatto (1-5)	Stato	Aperto il
R-12	Risk	Il fornitore di hosting ha comunicato una finestra di manutenzione non pianificata nella settimana del rilascio	Luca (Tech Lead)	4	Aperto	03/06
R-13	Risk	Se non testiamo il carico prima del Black Friday, rischio di rallentamenti sotto picco di traffico	Giulia (QA Lead)	5	In mitigazione	05/06
A-07	Assumption	Stiamo assumendo che l'ambiente di staging resti disponibile fino a fine mese	Luca (Project Manager)	3	Da verificare	01/06
A-08	Assumption	Assumiamo che Sara fornisca il feedback sul design entro 3 giorni lavorativi	Luca (Project Manager)	2	Da verificare	04/06
I-04	Issue	Il servizio di invio SMS di terze parti restituisce errori 500 dal 06/06	Marco (Sviluppatore backend)	4	In corso	06/06
D-05	Dependency	Il rilascio del modulo pagamenti dipende dall'approvazione del team sicurezza, prevista per il 12/06	Luca (Tech Lead)	5	In attesa	02/06

Con soli 6 righe, un Project Manager junior può già farsi una domanda utile: **quali elementi con impatto alto (4-5) non hanno ancora un'azione di mitigazione chiara?** In questo estratto sono tre (R-13, I-04, D-05): sono i primi tre di cui parlare nel prossimo check-in con il cliente, prima che diventino la causa di uno slittamento non comunicato per tempo.

8.3 RACI: chi fa cosa (e chi deve solo saperlo)

Una delle domande più comuni — e più fonte di attriti — in un progetto è: **“chi decide su questo?”** o **“a chi tocca fare questa cosa?”**. La matrice **RACI** è uno strumento semplice per chiarirlo prima che diventi un problema, assegnando a ogni attività uno dei quattro ruoli:

- **R — Responsible** (Esecutore): chi materialmente **fa il lavoro**.
- **A — Accountable** (Responsabile finale): chi **risponde del risultato** davanti a tutti — è la persona che, se qualcosa va storto, deve rispondere. Per ogni attività dovrebbe essercene **uno solo**, per evitare ambiguità.
- **C — Consulted** (Consultato): chi viene **coinvolto per un parere** prima che la decisione/attività sia completata, con una comunicazione a doppio senso (gli chiedi qualcosa, lui risponde).
- **I — Informed** (Informato): chi viene **aggiornato dopo il fatto**, con comunicazione a senso unico (non partecipa alla decisione, ma deve saperlo).

Analogia: pensa a un ristorante durante il servizio. Il cameriere che porta il piatto al tavolo è il *Responsible*: esegue l'azione. Lo chef di cucina è l'*Accountable*: risponde della qualità del piatto che esce dalla sua cucina. Il sommelier viene *Consulted* se il cliente chiede un abbinamento particolare di vino. Il titolare del ristorante è *Informed*: alla fine della serata gli viene detto quanti coperti sono stati serviti, ma non ha partecipato a nessun piatto.

Esempio pratico: matrice RACI per l'attività “rilascio di una nuova funzionalità in produzione” — lo stesso rilascio del modulo pagamenti visto nel RAID Log di ShopFacile qui sopra. I ruoli della tabella restano generici (così puoi riusarla per qualsiasi progetto), ma nel team ShopFacile sono coperti così: Sviluppatore → **Ahmed**, Tech Lead/Project Manager → **Luca**, QA/Tester → **Giulia**, Cliente → **Sara**, in quanto voce del business.

Attività	Sviluppatore	Tech Lead	Project Manager	QA / Tester	Cliente
Sviluppo del codice	R	C	I	I	—
Code review	C	A / R	I	I	—
Test della funzionalità	I	C	I	R	—
Decisione go/no-go al rilascio	I	C	A	C	I
Esecuzione del rilascio (deploy)	C	R	I	I	—
Comunicazione al cliente dell'avvenuto rilascio	I	I	R / A	I	I

Qualche osservazione utile da tenere a mente quando costruisci una RACI:

- Per ogni riga ci dovrebbe essere **esattamente una A** (Accountable): se nessuno è accountable, nessuno risponde davvero del risultato; se ce ne sono troppe, si diluisce la responsabilità e nelle emergenze “si scaricano” a vicenda.
- Una stessa persona può essere sia R che A sulla stessa attività (come il Tech Lead nella code review, sopra), specialmente in team piccoli.
- Troppi “C” su un’attività la rendono lenta: ogni consultazione è un potenziale collo di bottiglia. Usa “C” solo dove il parere serve davvero.

Una RACI ben fatta chiarisce chi fa cosa lungo il percorso di un rilascio, ma non dice ancora **quando** quel rilascio conta davvero per chi guarda il progetto da fuori. È qui che entra in gioco il concetto di milestone.

8.4 Milestone: un traguardo, non una scadenza qualsiasi

Una **milestone** (pietra miliare) è un **punto significativo** nel percorso del progetto: segna il raggiungimento di qualcosa di rilevante, non semplicemente il passaggio di una data sul calendario.

Analogia: pensa a una gara ciclistica a tappe. Ogni tappa ha ovviamente una data di arrivo, ma alcuni punti del percorso sono **traguardi volanti** o **cime di montagna**: superarli non è solo “un altro chilometro fatto”, è un momento che tutti notano, che dà punteggio a parte, che segna davvero un salto di fase della corsa. Una scadenza qualsiasi è come un cartello chilometrico: utile per orientarsi, ma non cambia la narrazione della corsa. Una milestone è il traguardo di tappa.

La differenza pratica:

	Scadenza semplice	Milestone
Cosa segna	Il completamento di un compito operativo	Il raggiungimento di un obiettivo significativo per il progetto
Visibilità	Interna al team, spesso operativa	Comunicata anche a stakeholder esterni
Esempio	“Completare la migrazione del database di test entro venerdì”	“Ambiente di test pronto e validato dal cliente”
Conseguenza se manca	Si riorganizza il lavoro della settimana	Può avere impatto su comunicazioni contrattuali, pianificazione di altri team, aspettative del cliente

Esempi concreti di milestone in un progetto software: “fine della fase di analisi dei requisiti”, “prima release disponibile per un gruppo pilota di utenti (beta)”, “completamento dei test di sicurezza”, “go-live in produzione”, “primo mese di produzione senza incidenti critici”. Non sono attività in sé, ma **traguardi** che raccontano un salto di stato del progetto — per ShopFacile, il go-live del nuovo modulo pagamenti che abbiamo visto nel RAID Log e nella RACI è esattamente questo tipo di milestone: non un task tra tanti, ma un momento che Sara comunicherà anche al di fuori del team.

In un contesto Agile, le milestone convivono bene con gli sprint: uno sprint può contribuire a una milestone senza esaurirla, e una milestone spesso coincide con la fine di una serie di sprint (ad esempio, la fine di una release che raggruppa più sprint). Ma un traguardo così visibile porta con sé anche più da perdere se qualcosa va storto lungo il percorso — ed è proprio quello che il prossimo paragrafo insegna a gestire.

8.5 Rischio: identificare, valutare, mitigare

Un **rischio** è un evento incerto che, se si verifica, ha un impatto (di solito negativo, anche se esistono rischi “positivi”, cioè opportunità) sul progetto. Gestire i rischi non significa eliminarli tutti — è impossibile — ma **conoscerli in anticipo** e decidere consapevolmente cosa fare.

Il processo tipico ha tre fasi:

1. **Identificazione**: elencare cosa potrebbe andare storto. Si fa in team, spesso in un momento dedicato (es. durante la pianificazione di una release), con tecniche semplici come il brainstorming o rivedendo i rischi materializzati in progetti simili passati.
2. **Valutazione**: per ogni rischio, stimare **probabilità** (quanto è plausibile che accada) e **impatto** (quanto danno farebbe se accadesse). Spesso si usa una scala semplice (basso/medio/alto) e si visualizza in una matrice.
3. **Mitigazione**: decidere una strategia. Le opzioni classiche sono:
 - **evitare** il rischio (cambiare approccio per non incontrarlo);
 - **ridurre** probabilità o impatto (azioni preventive);

- **trasferire** il rischio (es. a un fornitore, con un'assicurazione contrattuale);
- **accettare** il rischio consapevolmente, se il costo di mitigarlo supera il danno potenziale, tenendo comunque un piano B pronto.

Diagramma 3

Diagramma 3

Esempio concreto: “rischio di ritardo per dipendenza da un altro team” — lo stesso rischio D-05 già anticipato nel RAID Log di ShopFacile.

- **Descrizione del rischio:** la nuova funzionalità di pagamento di ShopFacile richiede che il team di infrastruttura fornisca un nuovo ambiente sicuro entro due settimane; quel team ha già altri due progetti in corso.
- **Probabilità:** media-alta (il team esterno è oggettivamente sovraccarico, lo si vede dal loro backlog).
- **Impatto:** alto (senza quell'ambiente, il rilascio non può partire, e la data comunicata al cliente è a rischio).
- **Mitigazione:** Luca decide di chiedere una conferma scritta della data al team infrastruttura appena possibile (per trasformare l'assunzione in un impegno verificato); valuta con il team un piano B con un ambiente temporaneo meno performante ma disponibile prima; e si accorda con Sara per comunicare per tempo al cliente che esiste un rischio, invece di farlo scoprire a due giorni dalla scadenza.

Da notare come questo esempio collega direttamente rischio, assunzione e dipendenza — gli stessi concetti visti nel RAID Log: non sono strumenti separati, ma **lenti diverse sullo stesso terreno**.

8.6 Pianificazione in un contesto Agile: livelli, non un unico piano

Nel project management “a cascata” tradizionale, l'idea è pianificare tutto in anticipo, in un unico grande piano dettagliato. In un contesto Agile/DevOps si pianifica invece **a più livelli**, con dettaglio crescente man mano che ci si avvicina all'esecuzione — perché sappiamo che i dettagli lontani nel tempo cambieranno comunque, mentre quelli vicini devono essere affidabili.

I livelli tipici, dal più generale al più operativo:

1. **Roadmap:** la visione a lungo termine (mesi/trimestri/anno), con i grandi temi e obiettivi di business. Non contiene dettagli tecnici, ma direzioni: nella roadmap di ShopFacile che Sara porta agli stakeholder potrebbe leggersi “nel primo trimestre miglioriamo l'esperienza di checkout”, “nel secondo trimestre lanciamo l'app mobile”.
2. **Release Plan:** un livello più concreto, che raggruppa più sprint per arrivare a un rilascio significativo per gli utenti o il cliente. Qui iniziano a comparire funzionalità specifiche e stime di massima.
3. **Sprint Plan:** il piano del singolo sprint (di solito 1-4 settimane, come hai visto nella sezione su Scrum), con gli elementi di backlog selezionati e scomposti in attività concrete.
4. **Daily:** l'allineamento quotidiano del team (la daily standup), dove si sincronizza il lavoro del singolo giorno rispetto all'obiettivo dello sprint.

Diagramma 4

Diagramma 4

Il punto chiave da capire, come futuro Project Manager: **più ti allontani nel tempo, meno il piano deve essere dettagliato — e va bene così**. Una roadmap troppo dettagliata è quasi sempre sbagliata (perché il futuro lontano è incerto), mentre uno sprint plan troppo vago è inutile (perché il team deve poter lavorare su qualcosa di concreto già domani mattina). Il tuo compito non è “prevedere tutto”, ma tenere questi livelli **coerenti tra loro** e aggiornarli quando la realtà li smentisce.

Avere un piano a più livelli, però, non ti dice ancora se ShopFacile lo sta effettivamente rispettando giorno per giorno: per quello serve osservare l’andamento reale del team, non solo il piano scritto.

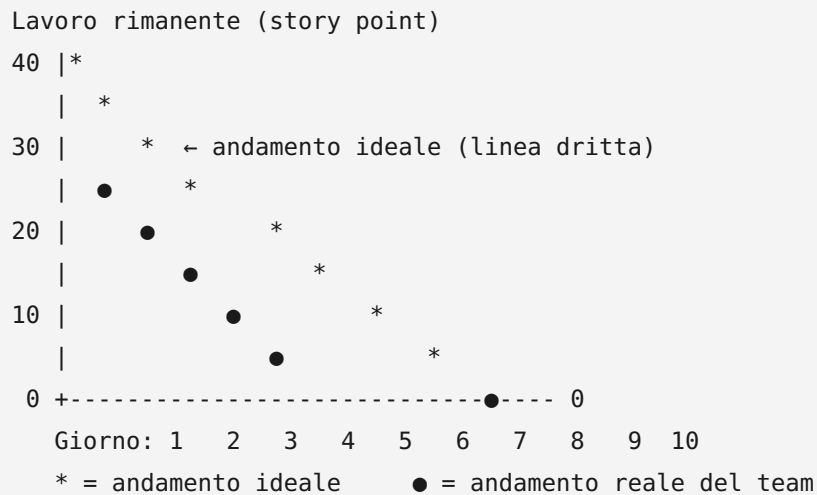
8.7 Monitoraggio: come si tiene traccia dell’andamento

Pianificare non basta: bisogna anche **verificare costantemente** se si sta procedendo come previsto, per poter reagire in tempo (non a fine progetto, quando è troppo tardi).

Gli strumenti principali di monitoraggio in un contesto Agile:

- **Daily stand-up**: il momento quotidiano (di solito 15 minuti, in piedi) in cui il team si sincronizza su cosa è stato fatto, cosa si farà e quali ostacoli ci sono. È monitoraggio “in tempo reale”, fatto dal team per il team.
- **Burndown chart**: mostra quanto lavoro **rimane** da fare in uno sprint (o in una release), giorno per giorno. La linea scende verso zero man mano che il lavoro si completa; una linea ideale tratteggiata mostra dove “dovremmo essere” per finire in tempo.
- **Burnup chart**: mostra quanto lavoro **è stato completato**, rispetto al totale previsto — utile soprattutto se lo scope cambia durante il progetto (con il burnup vedi anche se il “totale” sale, mentre il burndown da solo può confondere un aumento di scope con un rallentamento del team).
- **Dashboard**: pannelli visuali (spesso in Azure DevOps, Jira, o simili) che aggregano lo stato del backlog, delle Merge Request aperte, dei risultati delle pipeline di CI/CD, in un colpo d’occhio.

Un burndown chart “sano” tende ad avvicinarsi a una linea diagonale che scende gradualmente verso zero. Una rappresentazione testuale semplificata, per capire visivamente la forma:



In questo esempio, il team di ShopFacile parte più lento del previsto (la linea reale sta sopra quella ideale nei primi giorni — un possibile segnale di rischio da discutere in stand-up), ma recupera nella seconda metà dello sprint, chiudendo comunque a zero entro la fine. È tipicamente **Luca**, in quanto Scrum Master, a tenere d'occhio questo burndown e la dashboard del team ogni giorno: non guarda solo “se” la linea arriva a zero, ma **come** ci arriva — una discesa brusca solo nell'ultimo giorno è spesso il sintomo di lavoro “spuntato” tutto insieme a fine sprint, non di un reale progresso quotidiano.

Questi stessi grafici che Luca osserva ogni giorno sono anche la materia prima da cui si ricavano numeri di sintesi più stabili, utili per confrontare uno sprint con l'altro: i KPI.

8.8 KPI: gli indicatori chiave per un team Agile/DevOps

Un **KPI** (Key Performance Indicator, indicatore chiave di prestazione) è una misura quantitativa che aiuta a capire se le cose stanno andando bene, senza doversi fidare solo di impressioni soggettive (“mi pare che stiamo andando bene” non è un dato).

Alcuni KPI molto usati nei team Agile e DevOps:

KPI	Cosa misura	Perché è utile
Velocity	Quanti story point (o elementi di backlog) il team completa in media per sprint	Aiuta a pianificare quanto lavoro inserire negli sprint futuri, in modo realistico
Lead time	Tempo totale dalla richiesta di una funzionalità (apertura dell'elemento di backlog) al suo rilascio effettivo agli utenti	Misura quanto velocemente il team trasforma un'idea in valore consegnato
Cycle time	Tempo dall'inizio effettivo del lavoro su un elemento (non dalla richiesta) al suo completamento	Più mirato del lead time: misura l'efficienza del "fare", non dell'attesa in coda
Defect rate	Numero di bug rilevati (spesso in produzione) rispetto al volume di lavoro consegnato	Segnala la qualità di quanto viene rilasciato
Deployment frequency	Quante volte si rilascia in produzione in un dato periodo	KPI tipicamente DevOps: più alta è, di solito, più il processo di rilascio è maturo e automatizzato
Mean Time To Recovery (MTTR)	Tempo medio per ripristinare il servizio dopo un incidente in produzione	Misura la resilienza del team/ sistema, non solo la sua velocità
Sprint goal success rate	Percentuale di sprint in cui l'obiettivo dello sprint viene effettivamente raggiunto	Indica se la pianificazione degli sprint è realistica

Un avvertimento importante, da futuro Project Manager: i KPI vanno usati per **capire e migliorare il processo**, non per giudicare o "punire" le singole persone del team (es. confrontare la velocity di due sviluppatori diversi è quasi sempre un errore metodologico, perché la velocity dipende dal team nel suo insieme, dalla complessità del lavoro assegnato, e non è comparabile tra persone o addirittura tra team diversi). Usato male, un KPI distrugge la fiducia del team; usato bene, aiuta tutti a vedere dove migliorare.

Esempio pratico: confronto tra due sprint consecutivi del team di ShopFacile, con alcuni KPI di sintesi:

KPI	Sprint 8	Sprint 9	Cosa può significare
Velocity	32 story point	24 story point	Da solo non basta a dire se è un problema: va guardato insieme agli altri KPI
Lead time medio	6 giorni	9 giorni	Le richieste restano più tempo "in coda" prima di essere completate
Defect rate	2 bug su 15 elementi consegnati	5 bug su 11 elementi consegnati	Segnale di qualità in calo, non solo di velocità
Deployment frequency	4 rilasci in produzione	1 rilascio in produzione	Il processo di rilascio ha subito un rallentamento
Sprint goal success rate	Raggiunto	Non raggiunto	Conferma che qualcosa, nello sprint, non ha funzionato come previsto

Guardando la sola velocity, si potrebbe pensare "il team ha lavorato di meno". Ma leggendo la riga del defect rate e quella della deployment frequency insieme, l'ipotesi più plausibile è diversa: il team ha probabilmente **rallentato per gestire un numero maggiore di bug**, riducendo sia la velocity che la frequenza dei rilasci. Un Project Manager che si fermasse al primo numero (velocity in calo = "il team lavora peggio") trarrebbe la conclusione sbagliata; guardare i KPI **come insieme coerente**, non uno per uno isolato, è quello che permette di fare la domanda giusta in retrospettiva: "cosa ha causato l'aumento dei bug nello Sprint 9?", invece di "perché avete lavorato di meno?".

Questi KPI, però, restano numeri grezzi finché qualcuno non li traduce in un messaggio comprensibile per chi non li guarda ogni giorno: è il compito della reportistica.

8.9 Reportistica: cosa riportare, a chi, e con quale livello di dettaglio

Riportare lo stato di avanzamento è una delle attività più quotidiane di un Project Manager, ma non tutti i report sono uguali: **il livello di dettaglio deve adattarsi a chi lo legge**.

Analogia: pensa a come racconti la tua giornata a persone diverse. A un collega stretto racconti ogni dettaglio tecnico di un problema che hai risolto. A tuo padre, la sera, dici semplicemente "giornata impegnativa ma è andata bene". Nessuna delle due versioni è "più vera" dell'altra: sono adatte a interlocutori diversi, con bisogni informativi diversi.

Due macro-categorie di report, con caratteristiche molto diverse:

	Report per il management/ cliente	Report operativi per il team
Frequenza tipica	Settimanale o mensile	Quotidiana o per sprint
Livello di dettaglio	Alto livello: stato generale, rischi principali, milestone raggiunte o a rischio	Dettagliato: singoli task, blocchi tecnici, chi sta facendo cosa
Linguaggio	Orientato al business, senza gergo tecnico	Tecnico, con riferimenti diretti a task, PR, ambienti
Contenuto tipico	Avanzamento % rispetto alla roadmap, KPI di sintesi, rischi da RAID Log con impatto su tempi/costi	Burndown/burnup dello sprint, stato delle Merge Request, esito delle pipeline CI/CD
Obiettivo	Dare fiducia e visibilità decisionale a chi non segue i dettagli ogni giorno	Coordinare il lavoro quotidiano e risolvere blocchi in tempo reale

Esempio pratico: la stessa informazione — “il rilascio della nuova funzionalità di pagamento di ShopFacile slitta di una settimana per un problema di sicurezza emerso nei test” — viene comunicata in due modi molto diversi:

- **Al cliente/management, a cura di Sara:** “Il rilascio della funzionalità di pagamento è posticipato di una settimana per garantire il rispetto degli standard di sicurezza richiesti. Non ci sono impatti sul budget. Nuova data prevista: [data].”
- **Al team, in stand-up o nella dashboard operativa, a cura di Luca:** “Il test di sicurezza automatizzato ha rilevato una vulnerabilità nella gestione dei token di sessione nel modulo di pagamento (vedi issue #128); serve un fix nel branch `fix/token-sicurezza` prima di riaprire la Merge Request e far ripartire la pipeline.”

Stessa realtà, due livelli di risoluzione informativa. Un errore comune dei Project Manager junior è portare al cliente il livello di dettaglio tecnico del team (che genera confusione e ansia inutile), oppure, al contrario, dare al team solo informazioni di alto livello (che non bastano per lavorare). Imparare a “tradurre” tra questi due livelli è una delle competenze più preziose del ruolo.

8.10 Riepilogo

In questa sezione hai visto come i concetti di project management si inseriscano in un contesto Agile/DevOps, senza contraddire ciò che hai imparato su Scrum e Kanban, ma completandolo con una vista più ampia sul progetto nel suo insieme:

- gli **stakeholder** ti dicono chi coinvolgere e come;
- il **RAID Log** ti dà un metodo per non farti sorprendere da rischi, assunzioni non verificate, problemi e dipendenze;
- la **RACI** chiarisce chi fa cosa, evitando ambiguità e conflitti;

- le **milestone** segnano i traguardi che contano davvero, distinti dalle scadenze operative quotidiane;
- la gestione del **rischio** ti allena a pensare in anticipo, non solo a reagire;
- la **pianificazione a più livelli** (roadmap, release plan, sprint plan, daily) convive naturalmente con l'incertezza tipica di un progetto Agile;
- il **monitoraggio** (stand-up, burndown/burnup, dashboard) ti permette di intercettare deviazioni mentre sono ancora piccole;
- i **KPI** ti danno numeri oggettivi su cui basare le decisioni, se usati con criterio;
- la **reportistica** ti insegna che comunicare bene significa adattare il dettaglio a chi ascolta, non ripetere sempre lo stesso messaggio.

Nelle prossime sezioni vedrai come molti di questi concetti si traducano in strumenti concreti: la cultura DevOps (sezione 9) e la piattaforma Azure DevOps (sezione 10) offrono board, dashboard, work item e pipeline che mettono in pratica esattamente ciò che hai visto qui — RAID Log, RACI, burndown chart e KPI diventano funzionalità cliccabili in uno strumento reale.

Esercizi pratici

Gli esercizi che seguono ti aiutano a passare dal “ho capito il concetto leggendolo” al “saprei applicarlo su un progetto reale”. Puoi farli con carta e penna, un foglio di calcolo, o uno strumento online gratuito: lo strumento conta meno del ragionamento che ci metti dietro.

1. **Mappa gli stakeholder di un progetto immaginario.** Scegli un progetto software semplice (es. “un’app per prenotare turni in palestra”) ed elenca almeno 6 stakeholder diversi. Per ciascuno, assegna una posizione sulla matrice potere/interesse (come quella della sezione 8.1) e scrivi in una riga **come** li terresti aggiornati (es. “email mensile”, “invito al Sprint Review”, “coinvolgimento diretto nelle decisioni”). **Come verificare:** se per almeno uno stakeholder non sai dire “cosa vuole” e “come lo tieni informato” in una frase, la mappatura non è ancora abbastanza concreta — torna a rileggere la sezione 8.1.
2. **Costruisci un RAID Log di 8 righe per un progetto reale o immaginario.** Deve contenere almeno 2 rischi, 2 assunzioni, 2 issue e 2 dipendenze, ciascuna con una stima di impatto (scala 1-5) e un responsabile assegnato. **Come verificare:** prova a ordinare le righe per impatto decrescente. Se le prime tre righe della lista non hanno ancora un’azione di mitigazione scritta, il RAID Log è incompleto — un RAID Log senza azioni è solo un elenco di preoccupazioni, non uno strumento di gestione.
3. **Disegna una matrice RACI per un’attività che conosci.** Scegli un’attività diversa da quella già usata come esempio nella sezione 8.3 (ad esempio: “pianificazione di uno sprint”, “gestione di un incidente in produzione”, “onboarding di un nuovo membro del team”) e costruisci una tabella RACI con almeno 5 righe/attività e almeno 4 ruoli. **Come verificare:** conta le “A” in ogni riga. Se una riga ha zero A o più di una A, correggila: è l’errore più comune di chi costruisce una RACI per la prima volta.

4. **Calcola e interpreta due KPI su dati inventati.** Immagina un team che, in 4 sprint consecutivi, completa rispettivamente 28, 25, 30 e 18 story point, con 1, 2, 1 e 6 bug rilevati in produzione nello stesso periodo. Calcola la velocity media dei primi tre sprint e confrontala con il quarto; scrivi in 3-4 righe quale ipotesi formularesti su cosa è successo nel quarto sprint, motivando la risposta con i dati (non solo con un'impressione). **Come verificare:** la tua ipotesi deve citare **entrambi** i KPI (velocity e defect rate) insieme, non uno isolato — se la tua conclusione si basa solo sul calo della velocity, rileggi l'avvertimento sui KPI della sezione 8.8.
5. **Scrivi due versioni dello stesso aggiornamento di stato.** Pensa a un contrattempo plausibile in un progetto software (un bug bloccante trovato tardi, un ritardo di un fornitore, un test di sicurezza fallito) e scrivi due comunicazioni brevi (massimo 3-4 righe ciascuna): una per il cliente/management, una per il team in stand-up. **Come verificare:** fai leggere la versione “per il cliente” a qualcuno senza background tecnico (anche un amico non del settore): se non capisce cosa sta succedendo e cosa aspettarsi, è ancora troppo tecnica.
6. **Simula una retrospettiva sui rischi di un progetto concluso (anche immaginario).** Elenca 3 rischi che, con il senno di poi, si sono effettivamente concretizzati in issue, e per ciascuno scrivi: come lo avresti potuto identificare prima, e quale delle quattro strategie di mitigazione (evitare, ridurre, trasferire, accettare) avrebbe avuto più senso applicare in anticipo. **Come verificare:** per ogni rischio, la strategia di mitigazione che scegli deve essere diversa da un generico “fare più attenzione” — deve essere un'azione concreta e verificabile (es. “richiedere conferma scritta entro una data”, non “stare più attenti alle dipendenze”).

Collegamenti

- [9. DevOps](#) — la cultura e le pratiche che rendono possibile un monitoraggio continuo e KPI come deployment frequency e MTTR
- [10. Azure DevOps](#) — dove RACI, RAID Log, burndown chart e dashboard diventano strumenti concreti da usare ogni giorno

Risorse

- [PMI — Project Management Institute](#)
- [PMBOK Guide — panoramica](#)
- [Atlassian — Guida ai KPI Agile](#)
- [Atlassian — Cos'è un RACI Chart](#)
- [Atlassian — Stakeholder analysis](#)
- [Atlassian — Burndown chart](#)
- [Scrum.org — Risk Management in Scrum](#)
- [Axelos — PRINCE2 e RAID Log](#)